

Reinforcement Learning — Handwritten Notes Transcription

Transcribed from 39 scanned notebook pages (IMG_20260601_091807 – IMG_20260601_092641)

Part 1: The Explore–Exploit Dilemma

The Lazy Programmer’s Artificial Intelligence & Reinforcement Planning

- **explore – exploit** dilemma
- Does an experiment have a huge reward? → **Markov** → exploit
- $\mu = \bar{X} = \frac{1}{N} \sum_{k=0}^N X_k \rightarrow$ best estimate

We do the experiment 1 million times for each non-trivial task:

1. How do we know what is a realistic sample size for the experiment? → collect ∞
 2. → doing the worst → #0 of times → **Huge loss** (unoptimal)
 3. We are measuring a success rate + permitting we are measuring a reward for a Gaussian
-

1st Solution: -Greedy

- ε is a small number; at each round we choose to **explore** or **exploit**
- If ε is too small → explore = 100 best
- If random $< \varepsilon \rightarrow$ explore; strategy will not be optimal
- If $n \rightarrow \infty \rightarrow$ -greedy allows comparing the finds sample size for the experiment, because all rates will converge
 - **find estimated value**

A/B Testing can be helpful at a predetermined time to test after t_0 (statistical significance found)

2nd Solution: Optimistic Initial Values

- True mean $\ll$ estimated value (high ceiling) → goes down
 - **Exploitation**: sample near then converges
 - Estimated sample start → **maximum** → true mean → converges
 - If you don’t explore a bandit, the plan will remain high → **more exploration**
 - It won’t be forced to explore the greedy policy + use optimistic values from last
-

3rd Solution: UCB1 — Upper Confidence Bounds

- Confidence bounds = UCB1
- Small sample → large confidence bounds; big sample → small confidence bound

Chebyshev–Hoeffding bound:

$$P(|\bar{X} - \mu| \geq \varepsilon) \leq 2e^{-2N\varepsilon^2}$$

- **Upper bound:** $\hat{\varepsilon} = \sqrt{\frac{2 \ln N}{N_j}}$

$$X_{\text{UCB}} = \bar{X}_j + \sqrt{\frac{2 \ln(N)}{N_j}}$$

Be greedy (but will respect to $X_{\text{UCB}-j}$): - N_j small $\rightarrow N_j \ll \ln(N) \Rightarrow \hat{\varepsilon}$ large $\rightarrow$ using only sample $\rightarrow$ you'll have large uncertainty - N_j large $\rightarrow$ you'll have large confidence $\rightarrow$ small interval

For $t = 1 \dots N$:

$$\text{argmax}[\bar{X}_j] = \text{argmax} \left[\bar{X}_j + \sqrt{\frac{2 \ln(n)}{N_j}} + \varepsilon_j \right]$$

- On being large $\rightarrow$ if $N_j \ll$ if $N_j \rightarrow$ very small, start $\rightarrow$ not chosen $\rightarrow \text{prob}(\text{start}, r=1 \mid y, \text{start}) = 0.8$
 - $\text{prob}(\text{safe}, r=0 \mid y, \text{start}) = 0.5$
 - $\text{prob}(\text{hit}, r = -1 \mid d, \text{start}) = 0.4$
 - $\text{prob}(\text{safe}, r = 0 \mid \text{duck}, \text{start}) = 0.6$
 - $\rightarrow$ all other probabilities conditional yield 0
-

Part 2: Estimating Bandit Rewards

Estimating Bandit Rewards

$$\bar{X} = \frac{1}{N} \sum_{k=0}^N X_k \quad C_l(0, 1) \neq \text{mean} \quad \leftarrow N \text{ flips}$$

Problem: Storing all history + updating requires recalculating

Solution:

$$\begin{aligned} X_n &= \frac{1}{N} \sum_{k=1}^N X_k = \frac{N-1}{N} \bar{X}_{n-1} + \frac{1}{N} X_N \\ &= \bar{X}_{n-1} + \frac{1}{N} (X_N - \bar{X}_{n-1}) \end{aligned}$$

4th Method: Bayesian Sampling / Thompson Sampling

- Confidence intervals $\rightarrow$ approximately Gaussian
- $\bar{X} \sim \mathcal{N}\left(\mu, \frac{\delta^2}{N}\right), \bar{X} \sim \mathcal{N}\left(\frac{1}{\sqrt{N}}(\mu, \delta^2)\right)$

What if μ is also a random variable? θ

- Parameters are fixed - Parameters are random $\rightarrow$ **Bayesian**

$$P(\theta|X) = \frac{P(X|\theta) \cdot P(\theta)}{P(\theta)} = \frac{\int P(X|\theta)P(\theta) d\theta}{\text{likelihood} \times \text{prior}}$$

- $\rightarrow$ **posterior distribution:** approximate it if you can't
 - Conjugate prior: special $p(x|a), p(a)$ pairs where we can solve analytically — i.e., integral
-

Coin Toss $\rightarrow$ Bernoulli $\rightarrow$ Prior of Bernoulli

- $X_{1\theta} \rightarrow B(p)$ — prior of Bernoulli
- $\text{Beta}(\alpha, \beta) \equiv \frac{\theta^{\alpha-1}(1-\theta)^{\beta-1}}{B(\alpha, \beta)}$ — normalising factor
 - α : reflect belief (prior)
 - β : belief of prior information
- Prior: $P(\theta) = \frac{\theta^{\alpha-1}(1-\theta)^{\beta-1}}{B(\alpha, \beta)} \leftarrow$ mean
- $(\alpha, \beta) \in \mathbb{R}^2$ — hyperparameters of the prior hyperparameters = 1
- $\theta \in [0, 1]$ — interior hyperparameters – 1

Posterior:

$$\begin{aligned}
 P(\theta|X) &= \prod_{k=1}^N \theta^{x_k=1} (1-\theta)^{1(x_k=0)} \frac{\theta^{\alpha-1}(1-\theta)^{\beta-1}}{B(\alpha, \beta)} \\
 &= \theta^{\alpha+\sum 1(x_k=1)-1} (1-\theta)^{\beta+\sum 1(x_k=0)-1} \\
 &\Rightarrow \alpha' = \alpha + \#1_A \quad \beta' = \beta + \#0_A
 \end{aligned}$$

Gaussian / Thompson Sampling — Data Likelihood

$$P(X|\theta) \text{ likelihood : } X \sim \mathcal{N}(\mu_0, \lambda^{-1}); \quad \mu \sim \mathcal{N}(m_0, \lambda_0^{-1})$$

Update μ and N :

$$\begin{aligned}
 P(\mu|X) &= \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{N\sigma^2}{2}(\mu-\bar{\mu})^2} \\
 &= e^{-\frac{N\sigma_0^2}{2}(\mu-\mu_0)^2} \leftarrow \text{posterior Gaussian} \\
 &\Rightarrow \frac{1}{\sqrt{2\pi}} \cdot \frac{1}{\sqrt{2\pi}} \sqrt{\frac{N}{\delta^2}} \cdot e^{-\frac{1}{2} \left(\frac{(x_t - \mu)^2}{\delta^2/N} \right)} \sqrt{\frac{N}{2\pi\delta^2}} \\
 &= \frac{1}{2\delta^2} \left[(N_0 + N)\mu^2 - 2(m_0\lambda_0 + \sum x_t)\mu + (N_0\mu^2 - 2N_0\mu) \right] \\
 &= \frac{1}{2} \left[(N_0 + N) \left(\mu - \frac{m_0\lambda_0 + \sum x_t}{N_0 + N} \right)^2 \right]
 \end{aligned}$$

Here we are looking for a Gaussian — it's a Gaussian!

Identity $\hat{\theta}$:

$$\hat{\theta} = \mu_0 + t_0 \hat{\Sigma} X_t \rightarrow \text{no} \rightarrow A_0 t^{\hat{\Sigma} N}$$

- $P(A|X)$ sampling from Bayesian posterior:
 - μ is a random variable $\rightarrow$ can sample from it
 - more confident $\rightarrow$ more sample $\rightarrow$ more exploitation $\rightarrow$ **exploitation**
 - rough $\rightarrow$ rare $\rightarrow$ **explore**
-

Part 3: The RL Framework

Sensors $\rightarrow$ Environment $\rightarrow$ Feedback $\rightarrow$ Neurons $\rightarrow$ Parameterising

- **Finite states:** The factor = $\{0, X, \text{empty}\} = 3 \times 3 \text{ grid} = 3^9 \text{ states}$

MDP loop: 1. Start in State $S(t)$ 2. Take action $A(t)$ 3. Receive reward $R(t+1)$: $(s, a, s') \equiv (5, a)$ helper

- A Tic-Tac-Toe board $\rightarrow$ has board pieces numbers
 - Player 2 pieces in a row $\rightarrow$ block $\rightarrow$ player's board $\rightarrow$ bad for win
 - Play 2 pieces in a row $\rightarrow$ add $\rightarrow$ bad to win
-

Episodes

- One run of the game
 - # of episodes = hyperparameter
 - Learning across multiple $\rightarrow$ episode increases performance task: 3 processors run $\rightarrow$ draw
 - $\rightarrow$ **episodic task** $\rightarrow$ end $\rightarrow$ **episodic problem**
 - cont. $\rightarrow$ stop $\rightarrow$ gradient (policy following)
 - Beyond a certain θ we can't stop by moving point
 - some life $\rightarrow$ pole by moving pole
 - The good reward to maintain $\rightarrow$ is better/worse:
 - In chess $\rightarrow$ like good is to win
 - sub-goals well $\rightarrow$ encourage multi-lined behaviour
-

Credit Assignment

- **Credit assignment** — if 10 miles were seen with your app correctly, who gets the credit for the success $\rightarrow$ who gets the credit for the success
- **Delayed rewards** — When to choose bigger later rewards? VS short-term rewards, the long run
- What value should we attribute to state A?

$$\text{Value(A)} = 0.5 \times 1 + 0.5 \times 0 = 0.5$$

Value function $\rightarrow$ partially future rewards

- $\text{Value(A)} = 1 \times 1$ — reaching A is just as good as reaching B
 - since like my move from A is to B
 - Rewards are immediate; prefer states that should receive max
-

Part 4: Markov Decision Processes

MDP (Markov Decision Process)

$$P(s'_t, s_t) = P(s_t | s_{t+1})^{X_{t-1} \dots X_{t-2}}$$

$$P(X_{t+1} | X_t) = P(X_t | X_{t-1}, \dots, X_2)$$

Policies:

$$\Pi = \Pi(a|s) \quad \Pi_1 : \begin{cases} R = 0 \\ R_{t-p} \end{cases} \quad \Pi_2 : \begin{cases} R = 0 \\ R_{t-p} \end{cases}$$

- $\Pi \rightarrow$ Optimal policies are **not unique**

Returns: $G(t) = \sum_{\xi=1}^{+\infty} R(t + \xi)$

Discounting future rewards — $|\delta| < 1$:

$$G(t) = \sum_{\xi=0}^{\infty} \delta^\xi R(t + \xi) \quad (\text{convergent})$$

State-Value Function

$$V(s) = \mathbb{E}[G \mid S_t = s] = \mathbb{E}[r + \delta V(s') \mid S_t = s]$$

$$V_\pi(s) = \mathbb{E}_\pi[G \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{\xi=0}^{\infty} \delta^\xi R(t + \xi + 1) \right]$$

Action-Value function:

$$Q(s, a) = \mathbb{E}[r + \delta V(s') \mid S_t = s, A_t = a]$$

Optimal:

$$Q^*(s, a) = \max_a \left[\mathbb{E} \left(Q = \frac{s, a}{1} \right) \right] = \mathbb{E}[R(t+1) + \delta V^*(s_{t+1}) \mid S_t = s, A_t = s]$$

$$= \mathbb{E}[R(t+1) + \delta V^*(s_{t+1})] \quad \forall s \in S, A_t = s$$

Bellman Equations (recursive)

$$V_{\pi}(s) = \sum_a \Pi(a|s) \cdot \sum_{s'} P(s', r|s, a) [r + \delta V(s')]$$

$$V(s) = \mathbb{E}[r + \delta V(s')]$$

Bellman Optimality Equations (one unique optimal value):

$$V^*(s) = \max_a \mathbb{E}[R(t+1) + \delta V^*(S_{t+1}) \mid S_t = s, A_t = a]$$

$$V^*(s) = \max_a \sum_{s', r} P(s', r|s, a) [r + \delta V^*(s)]$$

How to find V given π ? $\rightarrow$ finding Π^* and V^*

- $P(r|s) = \sum_{s' \in S} P(s', r|s, a)$
 - $P(s'|s, a) = \sum_r P(s', r|s, a)$
 - I is in state s in the future: $V(s) = \mathbb{E}[G|s]$ = the value of all future states (the value of all possible trajectories)
-

Law of Total Expectation & Implementation

$$E[E(X)] \equiv E(X)$$

$$E_{\pi}[R(t+1)|S_t = s] = \sum_a \Pi(a|s) \sum_{s'} \sum_r P(s', r|s, a) \cdot r$$

$$E_{\pi}[\text{Any}|S_t = s] = \sum_a \Pi(a|s) \cdot E[\text{Any}]$$

$$E_{\pi}[E(X|Y)] = E(X) \quad \Leftarrow \text{Law of Total Expectation}$$

$$E(E(G(t+1))|\text{Any}) = -\epsilon^{S_{t+1}=S'}$$

Implementing Π^* : 1. Greedily choose the action that leads to the best $V(s)$ 2. 1-step lookahead search
3. With $Q(s, a)$ choose diagram (over a) 4. $Q(s, a) \rightarrow$ store the brute-force search result $\rightarrow Q(s, a)$

Part 5: Dynamic Programming

I. Dynamic Programming — Iterative Policy Evaluation

$$V(s) = 0 \quad \forall s \in S \quad (\text{initially})$$

WHILE True: $\Delta = 0 - V(s) = \sum_a \Pi(a|s) \cdot P(s'|s, a)[r + \delta V(s')] - \Delta = \max(\Delta, |V_{\text{old}} - V(s)|)$ - if $\Delta < \text{threshold}$: break

→ **return** $V(s)$ (instead of looping)

→ We can use the updated $V(s)$ for faster convergence

→ **Policy evaluation** → control problem

→ **Control** → Π^* (policy improvement)

Finding $V(s)$: initial value (V_0)

I. Finding Π → for I in range (max iteration): update $V(s)$

II. Finding Π^*

→ print (change in $V(s)$ vs limit / policy)

II. Policy Improvement / Policy Iteration

- Initialise value function and policy
- for $k = \infty \rightarrow$ large (non-Markovian reward):
 - state, action, newrewards → play game (policy)
 - update value function & Π

Find policy (non- Π): - if $|A| < \infty \rightarrow$ find $a \in A \rightarrow$ s.t. $Q_{\Pi}(s, a) > Q_{\Pi}(s) \leftarrow$ optimal policy Π - if $|A| = \infty$: $\Pi' \geq \Pi$

$V_{\Pi}(s) \leq V_{\Pi'}(s) \leftarrow$ indicates non-convergence (indicates convergence)

$$\Pi'(s) = \operatorname{argmax}_{Q_{\Pi}}(s, a) = \operatorname{argmax}_a \sum_{s', \pi} P(s', a|s, a)[r + \delta V_{\Pi}(s')]$$

→ **Value** V

Policy Iteration Convergence Problem

- Policy changing → the policy stops changing
- All of the values stop changing
- → value starts changing → value stops changing: important
- Alternating between **policy evaluation** and **policy improvement** (Back-tracking)

$V(s)$ = iterations + returns ($V(s)$) and policy $\Pi(s)$

1. Randomly initialise $V(s)$ and policy Π
2. $V(s)$ = iterations → (Bootstrapping + $\delta V(s)$)
3. policy-change = False — for all states:
 - old = policy(s);

- $\text{policy}(s) = \text{argmax}_a Q_\Pi(s, a)$
- if $\text{policy}(s) \neq \text{old}$: $\text{policy-changed} = \text{True}$
- if $\text{policy-changed} \rightarrow$ go back to 2

Disadvantage: slow — waiting for convergence (exploitation)

$\rightarrow$ winning $\rightarrow$ **policy iteration** improvement

Value Iteration — 2nd Solution

Value iteration = 2nd solution to find Bellman:

The necessarily greedy policy wouldn't change before V converges

$\Rightarrow$ Key: No need to work policy evaluation

$\Rightarrow$ when does policy iteration?

$$V_{k+1}(s) = \max_a \sum_{s', r} P(s', r | s, a) [r + \delta V_k(s)]$$

- **Online Evaluation** and important policy: calculating the policy **explicitly**
 - doesn't read V for $(k+1)$ -th value
 - $\rightarrow$ calculating $V = V_\Pi$ **doesn't need** finish calculation
 - $\rightarrow$ **policy iteration** — disadvantage: the whole environment from the environment
 - **Bootstrapped value** (diagram): $V = V_\Pi$, $V^* = V_\Pi^* \rightarrow$ policy iteration
 $\rightarrow$ policy iteration $\rightarrow$ the world $\rightarrow$ **policy iteration** — improvement
-

Part 6: Monte Carlo Methods

II. Monte Carlo Methods

(page 092454 — sparse, rest on back pages)

Episode: $G = \lambda(1 + \gamma^1 + \gamma^2 + \dots) = \lambda(t+1) + \gamma b(t+1)$

Monte Carlo $\rightarrow$ First Visit: 1. Every visit method: $t = 1 \times t = 3$ — are samples 2. Moving average (not recursion) when looping over - values are only updated for visited states - only updating from “start” — more states will never be visited - $\leftarrow$ “**Exploring Starts**” method

b) Only if 0.5 probability: $\varepsilon = 0$ — cannot policy

- Θ we don't know the action that leads to a better $V(s)$

Use Exploring Starts — to explore all $|S| \times |A|$ space action

- $|S| \times |A|$ for each action (instead of $\frac{|A|}{|S|}$ — state actions)

- $\leftarrow$ random initial $\rightarrow$ (random actions)

Remove Exploring Starts (if we can't: is too good $\rightarrow$ more blocking bad position):

$\rightarrow$ the -greedy = Π sometimes blocks

1. **Monte Carlo** — Cost to obtain samples and evaluate
2. **Improvement** = argmax over a :

$$Q(s, a) = \Pi(a|s, Q)$$

Best policy:

$$\Pi_b(a) = \operatorname{argmax}(Q)$$

The epsilon-greedy = Π — sometimes blocks

Part 7: Approximation Methods

Approximation Methods

1. = not useful until now $\rightarrow$ of all states and actions:

- estimate V for all S
- estimate V for all states and actions

2. **Neural network** $\rightarrow x = \text{state} \rightarrow x = \xi(a)$

- **Goal:** $V(s) = P(x; \theta)$ — differentiable

**Linear $\downarrow$ deep learning
approximation**

3. Can't use it now will use first:

- Linear models are not very expressive $\rightarrow$ feature engineering
 - **I states:** $V(s)$ — estimate $V(s)$ gives... (1 predict)
 - **II. TD(0) prediction**
 - **III. SARSA** $\rightarrow$ replace Q with linear function + control $\rightarrow$ approximator
-

Linear Model for RL

- $V(s)$ for $k \in \mathbb{R}$
- $G(s) \in \mathbb{R}$
- We will be doing regression; $\downarrow$ Monte Carlo $\downarrow$ sample mean

$$\text{Error} = [E[G|s_1, s_2 \dots s_T] - V(s)]^2 = [I \cdot G_{N+s} - V(s)]^2$$

- Treat G_n as a Moving sample $\rightarrow$ minimise the (individually squared) differences simultaneously
- $e = \sum_i (\hat{y}_i - y_i)^2$ — $\hat{y}_i$: 1 sample $\rightarrow$ 1 gradient decent
 - at each step after we only have to look at
 - a learning note

$$\theta = \theta + \alpha \cdot (G - \hat{V}(s))$$

$$\hat{V}(s, \theta) = \theta \cdot e(s) = \theta^T \cdot \nabla_{\theta} \hat{V}(s, \theta) \times$$

$$\nabla_{\theta} \hat{V}(s, \theta) = x$$

$$\theta = \theta + \alpha (G - \hat{V}(s, \theta)) \times \nabla_{\theta} \hat{V}(s, \theta)$$

Feature = creating categories g :

- $\rightarrow$ category 1 $\rightarrow$ feature creating ($\varepsilon_{(s)}$) for each state
- $\rightarrow$ category 2
 - $(x, y) \in \mathbb{R}^2 \rightarrow$ scale by $n = 0, \dots, 3^2$ — will a x (x_1, x_2) $\neq (x, j)$
 - The linear model is **not always** decreasing — **increasing** in monotone
 - **Solution** $\rightarrow$ create polynomial $\rightarrow$ Taylor expansion to approximate any function $\rightarrow$ for it works!

(Stop not parading): $\hat{V}(s) = \hat{V}(s) + \alpha \cdot (G_s - \hat{V}(s)) \cdot \frac{\partial \hat{V}(s)}{\partial V(s)}$

Feature $\rightarrow$ ($\hat{V}_{n-1} = \hat{V}_n$ — equivalent):
 $V(s)$ is not parameterised

Part 8: Temporal Difference (TD) Methods

II. TD(0) Approximation

$$G \neq TD(0)$$
$$G = r + \delta V(s')$$

$$\theta = \theta + \alpha(r + \delta \hat{V}(s') - \hat{V}(s, \theta)) \times \nabla_{\theta} \hat{V}(s, \theta)$$

- Target represents neural model prediction
 - This is called **semi-gradient descent**
-

2nd Technique for Solving MDPs: Temporal Difference Learning (TD)

- Fully online Bootstrapping
 - ($MC \neq$) completing (θ) $\rightarrow$ anything
 - [$MC \#$]: $\mathcal{E}(N) \rightarrow$ completing $\rightarrow$ {Dynamic programming}

Finding V given II: TD(0) | Hyperparameters | Bootstrapping

- approximates V - $\cdot$ proportion of long form in each state
- Control families: **SARSA**

On average finding:

$$V(S_t) \leftarrow V(S_t) + \alpha \cdot [G(t) - V(S_t)]$$

1. Exponential decay
2. Moving average

Q-Learning: — Bootstrapping:

$$V(S_t) \leftarrow V(S_t) + \alpha \cdot [G(t) - V(S_t)]$$

On average finding:

SA $\equiv$ ID $\times \frac{x}{(s,a)}$ = workaround $x \rightarrow$ unique x
 $\rightarrow$ word $\rightarrow$ unique $x = [\dots]$

Goal = find optimal Π : find optimal hyperparameters

- a model hyperparameters: 1 to 2 $\rightarrow$ learn
 - learning rate decreases at different rates
 - : decreases at a different rate
-

III. SARSA — non-greedy

- approximate $Q = \theta + \alpha(A + \delta\hat{Q}((\text{next})) - \hat{Q}((\text{next})))^2/2n$

Feature: $(m, n) \rightarrow$ new plane

$G = \theta + \alpha \times (A + \delta C, D, L, R)$

$x = (c, e, NC, UD, L, R)$

Batter features for each state

$x = [8 \times 8 = 64 \text{ features, one of } d \text{ — append }] \text{ — features for } |A| \text{ actions}$

After 3 actions $\times 4 \dots$ append if

After 6 states $\times 4 \text{ actions} = 36$

Deep Reinforcement Learning

Advanced AI: Deep Reinforcement Learning

- Agent: State $\rightarrow$ **Input layer** $\rightarrow$ DNN $\Pi_\theta(S|a) \rightarrow$ parameters $\rightarrow$ Action $\rightarrow$ Environment
 - Reward
 - Observed state

Controls: - II. N-step method - I. TD(A) - Policy Gradients - F. Network

Actor/Brain function:

- R, B, F $\rightarrow$ N - closely related SVM and NN

II. 1 — Controls with bias

End of transcription — 39 pages